Manual de Mantenimiento — Asistencia e Inventario

Plataforma InfoUNSA · Para el administrador de sistemas · Windows Server 2019+ / Windows 10-11 (con notas para Docker)

Este manual describe cómo mantener en operación las APIs de Asistencia e Inventario, sus bases de datos MySQL y las aplicaciones de escritorio de los kioscos: verificaciones periódicas, respaldos, actualizaciones, gestión de cuentas y resolución de problemas. La instalación está en los manuales 01 y 02.

Contenido

1. Mapa del sistema
2. Plan de mantenimiento periódico
3. Verificación del estado
4. Logs
5. Respaldos y restauración
6. Actualización de versión
7. Gestión de cuentas y seguridad
8. Mantenimiento de las bases de datos
9. Mantenimiento de los kioscos (aplicaciones de escritorio)
10. Problemas frecuentes
11. Recuperación ante desastres
12. Registro de mantenimiento
13. Anexo A. Estructura del repositorio

---

1. Mapa del sistema

Componente	Detalle
API de Asistencia	Puerto **43000** · servicio `AsistenciaAPI` (NSSM) o tarea `InfoUNSA-Servidores` · carpeta `apps\asistencia\api`
API de Inventario	Puerto **43001** · servicio `InventarioAPI` (NSSM) o tarea `InfoUNSA-Servidores` · carpeta `apps\inventario\api`
Bases de datos	MySQL 8: `infounsa_asistencia` e `infounsa_inventario` (servicio `MySQL80`, puerto 3306 solo local)
Configuración	Archivo `.env` dentro de `apps\<modulo>\api` (contiene contraseñas y secretos: **respaldarlo y protegerlo**)

Componente	Detalle
Logs de las APIs	`apps\<modulo>\api\logs\` (Winston) y `.run\logs\` (salida del servicio)
Clientes de escritorio	Instalados en cada kiosco. Datos locales en `%APPDATA%\infounsa-asistencia-desktop` y `%APPDATA%\infounsa-inventario-desktop`

Los kioscos trabajan **sin conexión** y sincronizan por lotes con la API; una caída de la API no detiene la marcación, pero hay que restablecerla para que los datos lleguen al servidor central.

---

2. Plan de mantenimiento periódico

Frecuencia	Tarea	Cómo
Diaria	Comprobar que ambas APIs responden	Sección 3 (`/health` de cada API)
Diaria	Confirmar que el respaldo de la noche se creó y no está vacío	Carpeta de respaldos (sección 5)
Diaria	Revisar errores del día	`error-AAAA-MM-DD.log` (sección 4)
Semanal	Revisar la cola de sincronización de los kioscos	Pestaña *Cola de Sync* de cada kiosco: debe estar en 0; ver problemas frecuentes
Semanal	Comprobar espacio libre en disco (servidor y kioscos)	`Get-PSDrive C` — alertar por debajo del 15 %
Mensual	Optimizar tablas de MySQL	Sección 8
Mensual	Probar la **restauración** de un respaldo en un equipo de prueba	Sección 5.3
Mensual	Revisar vulnerabilidades de dependencias	`pnpm audit` en la raíz del repositorio
Mensual	Comprobar hora del sistema en servidor y kioscos	`w32tm /query /status`; corregir con `w32tm /resync` (una hora desfasada altera las marcas)
Trimestral	Cambiar secretos y contraseñas	Sección 7
Trimestral	Revisar cuentas de usuario activas (dar de baja las que ya no se usan)	Gestión de operadores en la aplicación
Cada actualización	Respaldo previo, actualización y prueba	Sección 6

---

3. Verificación del estado

Servicios y puertos (PowerShell):

```
Get-Service MySQL80, AsistenciaAPI, InventarioAPI          # método NSSM
Get-ScheduledTask -TaskName InfoUNSA-Servidores            # método tarea programada
Get-NetTCPConnection -LocalPort 43000, 43001 -State Listen
```

Salud de las APIs — deben responder `status: ok` y `database: connected`:

```
Invoke-RestMethod http://localhost:43000/health
Invoke-RestMethod http://localhost:43001/health
```

Respuesta	Significado
`status: ok`, `database: connected`	Todo correcto
HTTP 503, `database: unavailable`	La API está viva pero no llega a MySQL: revisar `Get-Service MySQL80`, usuario y contraseña de `DATABASE_URL`
Sin respuesta	API detenida: `Start-Service AsistenciaAPI` (o `Start-ScheduledTask InfoUNSA-Servidores`) y revisar los logs

Desde otro equipo de la red (los kioscos): `Test-NetConnection <ip-del-servidor> -Port 43000`. Si falla y en el servidor responde, falta la regla de firewall o hay un bloqueo de red.

---

4. Logs

4.1. Dónde están

Log	Ubicación	Contenido
Aplicación	`apps\<modulo>\api\logs\application-AAAA-MM-DD.log`	Eventos `info`, `warn` y `error` en formato JSON
Errores	`apps\<modulo>\api\logs\error-AAAA-MM-DD.log`	Solo errores y excepciones
Salida del servicio	`.run\logs\<modulo>-api.log` y `.err.log`	Salida estándar y de error del proceso (arranque, fallos antes de que exista el log de la aplicación)

La rotación es automática: un archivo por día, **máximo 20 MB** por archivo (si se excede se crea otro con sufijo) y **14 días de retención**; los más antiguos se eliminan solos. Los archivos de `.run\logs` no rotan con Winston (NSSM los rota a 10 MB si se instaló con el script del repositorio).

4.2. Cómo consultarlos (PowerShell)

```
# Seguir los errores del día en tiempo real
Get-Content apps\asistencia\api\logs\error-$(Get-Date -Format yyyy-MM-dd).log -Tail
50 -Wait

# Contar errores del día
(Select-String -Path apps\inventario\api\logs\error-$(Get-Date -Format yyyy-MM-
dd).log -Pattern '"level":"error"').Count

# Buscar un problema concreto en todos los logs disponibles
Select-String -Path apps\*\api\logs\*.log -Pattern
'ECONNREFUSED|deadlock|P2002|P2034'
```

Si en un día hay más de unas decenas de errores o aparecen errores nuevos tras una actualización, abre una incidencia con el fragmento del log y la hora.

---

5. Respaldos y restauración

5.1. Qué respaldar

Qué	Con qué frecuencia	Por qué
Bases `infounsa_asistencia` e `infounsa_inventario`	**Diaria** (más una antes de cada actualización)	Son los datos oficiales
Archivos `.env` de ambas APIs	Cada vez que cambien	Contienen la cadena de conexión y los secretos JWT; sin ellos hay que reconfigurar
Carpeta de instaladores entregada (`instaladores\`)	Cuando se genere una versión nueva	Reinstalar kioscos sin recompilar

Guarda los respaldos **en otro disco o equipo** distinto al del servidor, y conserva al menos 14 días.

5.2. Respaldo automático diario

El repositorio incluye `scripts\respaldar-bases.ps1`, que ejecuta `mysqldump` (con transacciones consistentes, rutinas y *triggers*), comprime cada base en un `.zip` y elimina los respaldos con más de 14 días.

Prueba manual (PowerShell como administrador, desde la raíz del repositorio):

```
$env:MYSQL_PWD = "<contraseña del usuario infounsa>"
powershell -ExecutionPolicy Bypass -File scripts\respaldar-bases.ps1 -Destino
D:\Respaldos\InfoUNSA `
  -MysqlDump "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe"
```

Debe crear `infounsa_asistencia-AAAAMMDD-HHMM.sql.zip` e `infounsa_inventario-AAAAMMDD-HHMM.sql.zip` y mostrar sus rutas.

Programarlo cada noche (por ejemplo a las 02:00), guardando la contraseña como variable de entorno del sistema o dentro de un script protegido con permisos solo para el administrador:

```
$accion = New-ScheduledTaskAction -Execute "PowerShell.exe" -Argument `
  '-NoProfile -ExecutionPolicy Bypass -Command "$env:MYSQL_PWD=''<contraseña>''; &
''C:\Plataforma\infounsa-asistencia-inventario\scripts\respaldar-bases.ps1'' -Destino
D:\Respaldos\InfoUNSA -MysqlDump ''C:\Program Files\MySQL\MySQL Server
8.0\bin\mysqldump.exe''"'
$disparador = New-ScheduledTaskTrigger -Daily -At 2:00am
Register-ScheduledTask -TaskName "InfoUNSA-Respaldo" -Action $accion -Trigger
$disparador -User "SYSTEM" -RunLevel Highest
```

> La contraseña queda visible dentro de la definición de la tarea: restringe el acceso a administradores, o usa un usuario MySQL exclusivo para respaldos con permisos de solo lectura sobre ambas bases (consulta los privilegios que requiere `mysqldump` para tu versión de MySQL).

Con Docker, el equivalente es:

```
docker exec asistencia-db mysqldump -u root -p --single-transaction --routines --
triggers infounsa_asistencia > asistencia_AAAAMMDD.sql
docker exec inventario-db mysqldump -u root -p --single-transaction --routines --
triggers infounsa_inventario > inventario_AAAAMMDD.sql
```

5.3. Restauración

1. **Detén las APIs** para que no escriban durante la restauración (`Stop-Service AsistenciaAPI, InventarioAPI` , o `scripts\detener-servidores.ps1`).
2. Descomprime el respaldo elegido (`Expand-Archive`).
3. Restaura sobre la base (vacía o existente; los respaldos incluyen `DROP TABLE` y recrean las tablas):

```
Get-Content D:\Respaldos\InfoUNSA\infounsa_asistencia-AAAAMMDD-HHMM.sql -Raw | &
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u infounsa -p
infounsa_asistencia
```

(En CMD: `mysql -u infounsa -p infounsa_asistencia < archivo.sql`.)

4. Arranca las APIs y comprueba `/health` y que los datos recientes aparecen en la aplicación.
5. Los kioscos reenviarán automáticamente lo que tengan pendiente; el servidor ignora los registros ya existentes (cada marca lleva un identificador único).

Prueba mensual: restaura el último respaldo en un equipo de pruebas (con otro nombre de base) y verifica que abre y que los totales coinciden. Un respaldo que nunca se restauró no está comprobado.

---

6. Actualización de versión

Antes: avisa a los operadores, haz un **respaldo manual** de las bases (sección 5.2) y anota la versión actual (`git log -1 --oneline`).

6.1. Servidor (APIs)

```
cd C:\Plataforma\infounsa-asistencia-inventario
git pull
pnpm install
npm run build --prefix apps\asistencia\api
npm run build --prefix apps\inventario\api
Restart-Service AsistenciaAPI, InventarioAPI
# Método tarea programada: Stop-ScheduledTask InfoUNSA-Servidores; Start-
ScheduledTask InfoUNSA-Servidores
```

Comprueba `/health` de ambas APIs y que el kiosco inicia sesión y sincroniza. **No ejecutes el seed** en una actualización (solo se usa en la instalación inicial).

Si algo falla (vuelta atrás): `git checkout <commit-anterior>`, repite `pnpm install` y las compilaciones, reinicia los servicios y, si hubo cambios en la base, restaura el respaldo previo.

6.2. Kioscos (aplicaciones de escritorio)

Ejecuta el instalador nuevo (`InfoUNSA-Asistencia-Setup-<versión>-x64.exe` o `InfoUNSA-Inventario-Setup-<versión>-x64.exe`) **sobre el existente**. Los datos locales (marcas o movimientos pendientes, sesión, configuración) se conservan; no hay que desinstalar ni borrar carpetas. Las versiones nuevas se generan como se describe en el `README.md` (sección *Generar instaladores de Windows*).

Actualiza primero un kiosco, verifica que sincroniza, y luego el resto.

---

7. Gestión de cuentas y seguridad

7.1. Contraseñas iniciales

La instalación crea el usuario `admin` con contraseña `admin123` (y `almacenero` en Inventario, `operador_ast` en Asistencia, con la misma). **Cámbialas en el primer ingreso** desde la gestión de operadores de la aplicación.

7.2. Restablecer una contraseña olvidada

- **Una cuenta de operador:** el administrador la restablece desde *Gestión de operadores* (botón *Editar*).
- **La cuenta `admin`** (si nadie puede entrar): ejecuta el seed con la opción de restablecimiento, en la carpeta de la API correspondiente. Esto vuelve a fijar la contraseña de las cuentas iniciales y **no borra datos**:

  ```
  cd C:\Plataforma\infounsa-asistencia-inventario\apps\asistencia\api     # o
  apps\inventario\api
  $env:NODE_ENV = "development"          # el seed no se ejecuta con
  NODE_ENV=production
  $env:SEED_RESET_PASSWORDS = "true"
  $env:SEED_ADMIN_PASSWORD = "<nueva contraseña, mínimo 8 caracteres>"
  npm run db:seed
  ```

  Los valores definidos en la sesión de PowerShell tienen prioridad sobre los del `.env`. Después, cierra la ventana (para que las variables desaparezcan) y cambia la contraseña desde la aplicación. **Nunca** definas `SEED_DEMO_DATA=true` en producción: esa opción borra las asistencias.

7.3. Rotación de secretos (trimestral)

1. Genera un valor nuevo largo y aleatorio: `[Convert]::ToBase64String((1..48 | ForEach-Object { Get-Random -Maximum 256 }))`.
2. Cambia `JWT_SECRET` (Asistencia) y `JWT_SECRET_INVENTARIO` (Inventario) en el `.env` de cada API y reinicia el servicio. **Efecto:** todas las sesiones abiertas se cierran y los usuarios deben iniciar sesión de nuevo.
3. Contraseña de MySQL: `ALTER USER 'infounsa'@'localhost' IDENTIFIED BY '<nueva>';`, actualiza `DATABASE_URL` en ambos `.env` y reinicia las APIs. Actualiza también la tarea de respaldo.
4. Guarda una copia actualizada de los `.env`.

7.4. Otras medidas

- Mantén abiertos en el firewall **solo** los puertos 43000 y 43001; MySQL (3306) no debe ser accesible desde la red.
- Aplica las actualizaciones de Windows en una ventana de mantenimiento y reinicia comprobando después que servicios y APIs arrancan solos.
- Revisa mensualmente `pnpm audit`; las correcciones de seguridad se aplican como una actualización de versión (sección 6).

---

8. Mantenimiento de las bases de datos

Mensualmente, con el servidor con poca carga (cliente de MySQL como `infounsa`):

```
USE infounsa_asistencia;
ANALYZE TABLE attendance, users;
OPTIMIZE TABLE attendance, users;

USE infounsa_inventario;
ANALYZE TABLE inventory_items, inventory_movements, audit_logs;
OPTIMIZE TABLE inventory_items, inventory_movements, audit_logs;
```

Tamaño de las bases (para planificar el disco):

```
SELECT table_schema AS base, ROUND(SUM(data_length + index_length) / 1024 / 1024, 1)
AS mb
FROM information_schema.tables
WHERE table_schema IN ('infounsa_asistencia', 'infounsa_inventario')
GROUP BY table_schema;
```

No borres filas de `attendance`, `inventory_movements` ni `audit_logs` para "liberar espacio": son registros de trazabilidad. Si el crecimiento preocupa, planifica un archivado con el equipo técnico.

---

9. Mantenimiento de los kioscos (aplicaciones de escritorio)

Qué	Dónde / cómo
Datos locales (Asistencia)	`%APPDATA%\infounsa-asistencia-desktop\` — `local_db.sqlite` (base cifrada con SQLCipher), `local_db.key`, `sync-auth-asistencia.json`, `window-state.json`

Qué	Dónde / cómo
Datos locales (Inventario)	`%APPDATA%\infounsa-inventario-desktop\` — `local_inventory_db.sqlite`, `local_inventory_db.key`, `sync-auth-inventario.json`
Dirección del servidor	Pestaña **Ajustes** de la aplicación (`http://<ip-del-servidor>:43000` o `:43001`)

- **Semanal:** revisar que la *Cola de Sync* esté en 0 y que el indicador de la cabecera diga **ONLINE**.
- **Espacio en disco:** mantener más del 15 % libre; la base local crece con las marcas aún no sincronizadas.
- **Hora del equipo:** debe estar sincronizada (`w32tm /resync`); las marcas usan la hora del kiosco.
- **Respaldo de un kiosco:** las marcas ya sincronizadas están en el servidor, no hace falta respaldarlas. Si el equipo se avería con marcas **pendientes**, copia toda la carpeta `%APPDATA%\infounsa-*-desktop`. La clave de cifrado está protegida por Windows (DPAPI) y solo funciona con la misma cuenta de usuario del mismo equipo: restaura la carpeta en el **mismo equipo y usuario**; en otro equipo la base no podrá abrirse.
- **Reinstalar un kiosco:** ejecuta el instalador sobre el existente. Solo si el equipo técnico lo indica y no quedan marcas pendientes, borra la carpeta de datos para una instalación limpia.

---

10. Problemas frecuentes

Síntoma	Causa probable	Qué hacer
Kiosco en **OFFLINE**	API caída, red o URL del servidor mal configurada	`Invoke-RestMethod http://<servidor>:43000/health` desde el kiosco; revisar firewall, servicio y la URL en *Ajustes*
Cola de Sync no baja a 0	Sin conexión con la API, sesión vencida o registros rechazados	Confirmar ONLINE; cerrar sesión y volver a entrar; si persisten registros con *Error* o *Conflicto*, revisar `error-*.log` del servidor y corregir la causa (datos inválidos, usuario inexistente, stock insuficiente)
Todos los usuarios deben iniciar sesión de nuevo	Se cambió el secreto JWT o el servicio se reinició sin `JWT_SECRET`	Verificar que el `.env` tiene el secreto definido y no cambia entre reinicios
`/health` responde 503	MySQL detenido o credenciales cambiadas	`Start-Service MySQL80`; comprobar usuario y contraseña de `DATABASE_URL`; revisar `.run\logs\*.err.log`
El servicio se detiene al arrancar	Falta `.env`, no existe `dist\main.js` o el puerto está ocupado	`.run\logs\*.err.log`; `npm run build`; `Get-NetTCPConnection -LocalPort 43000`

Síntoma	Causa probable	Qué hacer
Error `EADDRINUSE`	Otra instancia de la API (iniciada a mano) usa el puerto	`scripts\detener-servidores.ps1` o `Stop-Process -Id <PID>`
Marcas con hora incorrecta	Reloj del kiosco desajustado	`w32tm /resync`; revisar zona horaria
Stock "negativo" rechazado en Inventario	La regla de integridad impide salidas mayores al stock	Revisar el movimiento; registrar primero la entrada correspondiente
Disco lleno en el servidor	Respaldos acumulados o logs	Mover respaldos antiguos; los logs de las APIs se limpian solos a los 14 días
Tras actualizar, errores nuevos	Cambio de versión	Revisar `error-*.log`; si es grave, vuelta atrás (sección 6.1)

---

11. Recuperación ante desastres

Servidor perdido o dañado:

1. Prepara un servidor nuevo e instala los requisitos (MySQL, Node.js, pnpm, Git, NSSM) siguiendo los manuales 01 y 02, **hasta el paso e** (sin crear aún las tablas).
2. Restaura los `.env` guardados (o recréalos con los mismos secretos JWT para no cerrar sesiones).
3. Crea las bases y el usuario (paso c) y **restaura los respaldos** (sección 5.3) en lugar de ejecutar el esquema y el seed.
4. Compila, crea la regla de firewall y los servicios (pasos e, f, g).
5. Si el servidor nuevo tiene otra dirección IP, actualiza la URL en *Ajustes* de cada kiosco (o reutiliza la IP anterior).
6. Comprueba `/health` y que los kioscos sincronizan lo pendiente.

Base de datos dañada: detén las APIs, restaura el último respaldo bueno y arranca. Los datos posteriores al respaldo que estén en los kioscos se reenviarán; los registros hechos directamente en el servidor después del respaldo deberán recuperarse manualmente.

Kiosco averiado: instala la aplicación en un equipo de reemplazo, configura la URL del servidor e inicia sesión. Las marcas sincronizadas están a salvo en el servidor; para las pendientes, ver sección 9.

---

12. Registro de mantenimiento

Anota cada intervención para tener trazabilidad:

Fecha	Responsable	Equipo / sistema	Tarea realizada	Resultado	Observaciones

---

Anexo A. Estructura del repositorio

Este anexo indica qué contiene cada carpeta y archivo importante, para saber **dónde buscar o modificar** al dar mantenimiento. Las carpetas de dependencias (`node_modules`) y de compilación (`dist` , `dist-electron`) se generan al instalar y compilar; no se versionan.

A.1. Vista general

```
infounsa-asistencia-inventario/
├── apps/
│   ├── asistencia/
│   │   ├── api/              API de Asistencia (NestJS), :43000
│   │   ├── desktop/          Kiosco de Asistencia (Electron)
│   │   └── database/         Scripts SQL de referencia
│   ├── inventario/
│   │   ├── api/              API de Inventario (NestJS), :43001
│   │   ├── desktop/          App de Inventario (Electron)
│   │   └── database/init/    SQL del esquema y triggers
│   └── portal-general/       Página de enlace (opcional)
├── packages/
│   ├── shared-config/        MySQL, logs, guardas y roles
│   ├── shared-types/         Tipos TypeScript comunes
│   └── shared-ui/            Componentes y cliente HTTP
├── scripts/                  Servidores, servicios, respaldos
├── infra/                    Dockerfiles y configuración
├── windows-server-deploy/    Despliegue con contenedores (alt.)
├── qa/                       Pruebas de sistema y Postman
├── tools/                    Pipeline local y seguridad
├── docs/                     Guías de despliegue y operación
├── Manuales/                 Manuales (Markdown y PDF)
├── docker-compose.yml        Desarrollo/pruebas con Docker
├── docker-compose.prod.yml   Producción con Docker
├── .env.example              Plantilla de variables
├── package.json, pnpm-*.yaml  Monorepo y paquetes
└── README.md                 Inicio e instalación
```

A.2. API de Asistencia — `apps/asistencia/api/`

Ruta	Qué contiene
`src/main.ts`	Arranque de la API: puerto, CORS (`ASISTENCIA_ALLOWED_ORIGINS`), validaciones y logs
`src/app.module.ts`	Módulo raíz que registra todos los módulos
`src/modules/attendance/`	**Marcaciones:** registro, sincronización por lotes desde los kioscos, reportes PDF y Excel, DTOs de validación
`src/modules/auth/`	Inicio de sesión, emisión y validación de tokens JWT
`src/modules/users/`	Usuarios y personal (alta, edición, baja) y **carga masiva desde Excel**
`src/modules/sedes/` · `terminals/`	Sedes y terminales (kioscos)
`src/modules/barcode/`	Lectura y simulación de códigos de barras (página `barcode-scanner.html` de pruebas)
`src/modules/stats/`	Estadísticas en tiempo real para el panel (WebSocket)
`src/modules/health/`	Endpoint `/health` (estado de la API y de la base de datos)
`src/common/auth/`	Secreto JWT, guardas de autenticación y de roles
`src/common/database/`	Conexión a MySQL, cliente Prisma y reintentos de transacciones
`src/common/logging/`	Configuración de Winston: archivos de log, rotación 20 MB / 14 días
`prisma/schema.prisma`	**Esquema de la base de datos** (tablas y relaciones)
`prisma/seed.ts` · `data-usuarios.json`	Datos iniciales: roles, sede, terminal y usuarios `admin` y `operador_ast`
`prisma/migrations/`	Migraciones históricas (no se usan para instalar; ver manual 01)
`prisma.config.ts`	Configuración de Prisma (lee `DATABASE_URL`)
`scripts/test-mysql-integration.mjs`	Prueba de integración contra MySQL real
`*.spec.ts` (junto a cada archivo)	Pruebas unitarias (Jest): `npm test`
`.env` *(no versionado)*	Configuración local de la instalación

A.3. API de Inventario — `apps/inventario/api/`

Ruta	Qué contiene
`src/main.ts` · `app.module.ts`	Arranque de la API (puerto 43001, CORS) y módulo raíz

Ruta	Qué contiene
`src/inventory-items/`	**Catálogo de bienes:** alta, edición, búsqueda, código automático
`src/inventory-movements/`	**Movimientos de almacén:** entradas y salidas, control de stock, alertas de stock crítico, reportes PDF y Excel, sincronización desde los clientes
`src/categories/` · `areas/`	Categorías y áreas
`src/users/`	Usuarios y operadores
`src/auth/`	Inicio de sesión, JWT, guardas y roles
`src/health/`	Endpoint `/health`
`src/prisma/`	Conexión Prisma a MySQL
`src/config/`	Conexión MySQL y configuración de logs (Winston)
`src/middlewares/`	Registro de peticiones HTTP
`prisma/schema.prisma`	Modelo de datos usado por la API
`prisma/seed.ts`	Datos iniciales: usuarios `admin` y `almacenero`
`prisma/seed_30_inventarios.sql`	Datos de ejemplo de bienes (solo pruebas)
`../database/init/001_*.sql` · `002_*.sql`	**Creación real del esquema**, *triggers* y reglas de integridad (stock no negativo)

A.4. Aplicaciones de escritorio — `apps/asistencia/desktop/` y `apps/inventario/desktop/`

Tienen la misma organización:

Ruta	Qué contiene
`main.js`	**Proceso principal de Electron:** ventana, base local cifrada, comunicación con la API y motor de sincronización
`preload.js`	Puente seguro entre la interfaz y el proceso principal
`schema.sql`	Esquema de la base local SQLite cifrada (SQLCipher)
`desktop-config.json`	Configuración inicial (dirección del servidor)
`database.js` *(Asistencia)*	Utilidades de la base local
`reset-local-data.js` *(Asistencia)*	Solo desarrollo: borra los datos locales del equipo (`npm run reset:local-data`)
`src/main/persistence/`	Acceso a la base local (adaptador SQLite)
`src/main/security/`	Clave de cifrado local, sesión, contraseña sin conexión, validación de origen de mensajes

Ruta	Qué contiene
`src/main/sync/` *(Asistencia)*	Cliente de sincronización con la API
`src/App.tsx` · `main.tsx` · `App.css`	Interfaz React: pantalla principal y estilos
`src/hooks/`	Lógica de datos y sesión (`useAttendanceData` , `useInventoryData` , `useSyncQueue` , escáner de códigos)
`src/utils/`	Cliente HTTP y validadores
`src/types/`	Tipos TypeScript
`src/ErrorBoundary.tsx`	Captura de errores de la interfaz
`vite.config.ts` · `tsconfig.json`	Compilación de la interfaz
`package.json`	Dependencias y configuración de **electron-builder** (instalador NSIS)

Pantallas (`src/components/`):

Aplicación	Carpetas
Asistencia	`kiosk/` marcación de asistencia · `dashboard/` panel principal · `log/` registro del día · `sync/` cola de sincronización · `reports/` reportes · `admin/` carga masiva · `settings/` ajustes y operadores · `ui/` elementos comunes (modal, teclado táctil, tooltips, registro de visitantes) · `WebcamScanner.tsx` lector por cámara
Inventario	`dashboard/` panel principal · `catalog/` catálogo de bienes y carga desde Excel · `movements/` entradas y salidas · `alerts/` alertas de stock · `reports/` reportes · `operators/` usuarios · `settings/` ajustes · `ui/` elementos comunes

A.5. Bases de datos — `apps/*/database/`

Ruta	Qué contiene
`apps/inventario/database/init/`	Scripts SQL que **crean** el esquema de Inventario (se ejecutan en la instalación)
`apps/asistencia/database/init/`	Scripts SQL históricos de Asistencia (**no usar** para instalar: el esquema de Asistencia lo crea Prisma)
`apps/asistencia/database/sqlite-referencia/`	Referencia del esquema local SQLite y scripts de prueba en Python

A.6. Paquetes compartidos — `packages/`

Ruta	Qué contiene
`shared-config/src/database/`	Opciones de conexión a MySQL (incluye las variables `MYSQL_SSL*`)
`shared-config/src/logging/`	Configuración común de logs
`shared-config/src/auth/`	Decoradores y guardas de roles y de rutas públicas
`shared-types/src/`	Tipos y errores compartidos, y la definición de la API de Electron
`shared-ui/src/`	Notificaciones (`ToastProvider`), captura de errores y cliente HTTP

A.7. Scripts de operación — `scripts/`

Script	Para qué sirve
`iniciar-servidores.sh` / `.ps1` · `detener-servidores.sh` / `.ps1`	Iniciar y detener las dos APIs en segundo plano
`instalar-servicios-nssm.ps1`	Instalar las APIs como servicios de Windows con NSSM
`instalar-autoarranque.sh` / `.ps1`	Inicio automático al reiniciar (systemd en Linux, tarea programada en Windows)
`respaldar-bases.ps1`	Respaldo de las bases con `mysqldump` y limpieza de respaldos antiguos
`generar-instaladores-windows.ps1` / `.sh` · `build-desktop-installers.ps1`	Generar los instaladores `.exe` de los kioscos en `instaladores\`
`generar-manuales-pdf.py` · `manuales-a-pdf.js`	Generar los PDF de la carpeta `Manuales/pdf/`
`validate-all.ps1` · `run-inventario-cpi.ps1`	Validaciones y pruebas de integración locales
`publish-to-gitlab.ps1`	Publicación de instaladores en GitLab (uso interno del equipo)
`backup/` · `maintenance/` · `seed/` · `setup/`	Carpetas reservadas (vacías)

A.8. Infraestructura y despliegue

Ruta	Qué contiene
`docker-compose.yml`	Entorno de desarrollo/pruebas: bases MySQL, APIs, Adminer y portal general
`docker-compose.prod.yml`	Producción con Docker (imágenes publicadas por el pipeline)

Ruta	Qué contiene
`docker-compose.asistencia-inventario.qa.yml`	Entorno aislado para pruebas de calidad
`infra/docker/`	Dockerfiles de las APIs y del cliente de Inventario
`infra/nginx/` · `infra/compose/`	Configuración de Nginx y fragmentos de Compose
`windows-server-deploy/`	Despliegue alternativo con contenedores en Windows Server y registro de tarea de inicio
`start-backend-windows.ps1`	Inicio nativo de las APIs en Windows (con PM2 si está disponible)
`.gitlab-ci.yml`	Definición del pipeline de integración continua (GitLab)
`.env.example`	Plantilla de variables de entorno

A.9. Pruebas y herramientas — `qa/` y `tools/`

Ruta	Qué contiene
`qa/scripts-pruebas-de-sistema/`	Pruebas de sistema con Playwright (casos `cps-*` de Asistencia e Inventario) y generador de reporte Excel
`qa/postman/`	Colecciones Postman/Newman de pruebas de integración de las APIs y archivos de prueba (`fixtures/`)
`qa/inventario/`	Script para ejecutar pruebas de Inventario en Docker
`tools/dev/`	Pipeline local de validación (`run-local-ci.ps1` , `run-local-pipeline.ps1` , `validate-docker-asistencia-inventario.ps1`)
`tools/security/semgrep-rules.yml`	Reglas de análisis estático de seguridad

A.10. Documentación — `docs/` y `Manuales/`

Ruta	Qué contiene
`Manuales/01` y `02`	Manuales técnicos de instalación de cada API
`Manuales/03` y `04`	Manuales de usuario de Asistencia e Inventario
`Manuales/05`	Este manual de mantenimiento
`Manuales/Anexo-servicios-windows.md`	Detalle de servicios de Windows (NSSM y tarea programada)
`Manuales/pdf/` · `Manuales/img/`	Versiones PDF y capturas de pantalla

Ruta	Qué contiene
`docs/despliegue/`	Guías de despliegue en producción, migración a MySQL, Windows Server y vuelta atrás
`docs/operacion/`	Manual de usuario final, datos de prueba, respaldos y monitoreo (documentos de una versión anterior; **el procedimiento vigente es el de este manual**)
`docs/excel-ejemplos/`	Plantillas Excel de ejemplo para la carga masiva
`docs/Guía de instalación y despliegue*.md/.pdf`	Guías de instalación de cada API

Dónde tocar según la tarea:

Quiero…	Voy a…
Cambiar una regla de negocio de marcaciones o de stock	`apps/asistencia/api/src/modules/attendance/` o `apps/inventario/api/src/inventory-movements/`
Agregar un campo a una tabla	Esquema (`prisma/schema.prisma` ; en Inventario también `database/init/*.sql`) y la interfaz correspondiente
Cambiar un texto o una pantalla de una aplicación	`apps/*/desktop/src/components/<pantalla>/`
Cambiar cómo se sincroniza un kiosco	`apps/*/desktop/main.js` y `src/main/`
Ajustar permisos o roles	`packages/shared-config/src/auth/` y los controladores del módulo
Cambiar la rotación de logs	`src/common/logging/winston.config.ts` (Asistencia) o `src/config/winston.config.ts` (Inventario)
Ajustar el instalador (nombre, accesos directos)	`apps/*/desktop/package.json` , sección `build`
Cambiar puertos o dirección de la base de datos	`.env` de cada API (no se edita el código)